

Customizing the systemd Service

This learning resource follows [Getting Started for System Administrators](#). Review the host engine's ownership, credentials, trust boundary, and restart behavior there before changing its service configuration.

A typical DCGM installation runs *nv-hostengine* as *nvidia-dcgm.service*. In that arrangement, systemd—not the shell from which an administrator later runs *dcgmi*—creates the host-engine process, supplies its environment, monitors it, and decides whether to restart it.

This distinction matters whenever a DCGM setting must be present from process startup. Exporting a variable in an interactive shell changes the environment of commands launched from that shell. It does not change the environment of an already-running *nv-hostengine* that systemd launched earlier. Likewise, passing an option to a one-off *nv-hostengine* command does not change the instance managed by *nvidia-dcgm.service*.

Note

Systemd creates the supervised host-engine process and supplies its startup identity and environment. This process boundary is why a DCGM variable intended for the host engine must be configured in the service environment before the process starts; exporting it in a later login shell does not modify that running process. For the underlying process-inheritance model, see the POSIX [Environment Variables](#) and [exec](#) specifications. For the service environment and supervision model, see systemd's [Environment= documentation](#) and [systemd.service](#).

Common reasons to customize the service include:

- adding a host-engine option, such as a startup module denylist;
- changing the listener, log level, home directory, or child-process account;
- providing environment variables required by a diagnostic executable;
- adding ordering dependencies for other local services; or
- applying site-specific systemd resource or security controls.

The examples on this page use the canonical unit name *nvidia-dcgm.service*. The packaged unit also declares *dcgm.service* as an alias, but using the canonical name makes file locations and troubleshooting output unambiguous.

 Warning

Applying most service changes requires restarting *nv-hostengine*. A restart disconnects clients and discards process-owned state, including groups, field groups, watches, retained samples, jobs, module state, and health or policy configuration. Coordinate the restart and recreate the state required by long-running clients afterward.

See [HOST-ENGINE STATE AND RESTARTS](#) for the DCGM objects and effects that do and do not survive a restart.

How systemd Builds the Service

Systemd does not necessarily read one file and use it verbatim. It assembles an *effective unit* from a main unit file and zero or more drop-in files. The effective unit is the configuration that will be used the next time the service starts.

For DCGM, the relevant configuration layers are:

DCGM service configuration layers

Layer	Typical location	Purpose
Vendor main unit	<i>/usr/lib/systemd/system/nvidia-dcgm.service</i>	Defaults supplied and updated by the DCGM runtime package.
Administrator main unit	<i>/etc/systemd/system/nvidia-dcgm.service</i>	Optional complete replacement for the vendor main unit.
Generated runtime drop-ins	<i>/run/systemd/system/nvidia-dcgm.service.d/*.conf</i>	Transient settings generated for the current boot, including the multi-node diagnostics environment.
Administrator drop-ins	<i>/etc/systemd/system/nvidia-dcgm.service.d/*.conf</i>	Persistent local changes that are merged with the selected main unit.

A main unit in */etc* replaces the main unit in */usr/lib*. Drop-ins are a separate layer: they still apply whether the selected main unit came from */usr/lib* or */etc*. Consequently, installing a full replacement unit does not by itself suppress the multi-node diagnostics drop-in under */run*.

Systemd gathers applicable **.conf* drop-ins, resolves files with the same name in favor of the higher-priority directory, and then processes the remaining files in lexicographic filename order. For the system service paths used here, */etc* takes precedence over */run*, and */run* takes precedence over vendor configuration. This is why a site file named *90-mndiag-local.conf* can replace environment values set by the generated *50-mndiag.conf*.

Directive behavior also matters. A later scalar assignment usually replaces an earlier value. Some directives are lists and accumulate values unless they are explicitly cleared. *ExecStart=* must be cleared before the packaged command can be replaced in a drop-in.

Note

Unit-file precedence matters because an administrator file can replace a packaged unit while later drop-ins can still modify the selected unit. See [Unit File Load Path and drop-in behavior in systemd.unit](#).

Reload and restart are different operations

Two systemd operations that sound similar have different effects:

Configuration and process operations

Operation	What it changes	What it does not change
<i>systemctl daemon-reload</i>	Makes systemd reread unit files and rerun system generators.	Does not replace the running host-engine process or its environment.
<i>systemctl restart nvidia-dcgm.service</i>	Stops the current host engine and starts a new process from the effective unit.	Does not make the service start automatically on a future boot.
<i>systemctl enable nvidia-dcgm.service</i>	Installs the links that pull the service into a future boot.	Does not start the service immediately unless <i>-now</i> is also used.

When files are edited directly, the usual sequence is therefore:

1. edit the unit or drop-in;
2. run *systemctl daemon-reload* so systemd sees the change;
3. inspect the effective unit;
4. restart the service so a new process receives the change; and
5. make a DCGM request to verify actual service readiness.

systemctl edit and *systemctl edit -full* perform the daemon reload automatically after the editor exits successfully. They still do not restart the service.

Note

A daemon reload updates systemd’s configuration model; only a service restart creates a new host-engine process with that model, and only enablement affects future boots. See the [systemctl manual](#) for the exact operation semantics.

Understand the Packaged Unit

The packaged file is `/usr/lib/systemd/system/nvidia-dcgm.service`. It contains three kinds of information:

[Unit]

Describes the service and its ordering relationships with other units.

[Service]

Describes the process to start, its identity and environment, and restart behavior.

[Install]

Describes the links created when the service is enabled for boot.

The important packaged directives are:

Packaged `nvidia-dcgm.service` behavior

Setting	Why it matters
<code>After=nvidia-gpu-reset.target</code>	Orders DCGM startup after the NVIDIA GPU reset target when both are in the same transaction. <code>After=</code> controls order; it does not pull the target into the transaction by itself.
<code>Wants=nvidia-gpu-reset.target</code>	Requests the reset target when the DCGM service is started.
<code>User=root</code>	Runs the host-engine process as root so privileged DCGM functionality is available.
<code>PrivateTmp=false</code>	Gives the service the host’s normal temporary-directory namespace rather than a systemd-private <code>/tmp</code> .
<code>Environment="DCGM_HOME_DIR=/var/log/nvidia-dcgm"</code>	Supplies the working and default diagnostic-output directory before the host engine starts.

Setting	Why it matters
<i>ExecStart=/usr/bin/nv-hostengine -n --service-account nvidia-dcgm</i>	Runs the host engine in the foreground and selects <i>nvidia-dcgm</i> for supported diagnostic child processes.
<i>Restart=on-abort</i>	Requests a restart after an abort-style signal, subject to systemd's start-rate limits.
<i>WantedBy=multi-user.target</i>	Makes the enabled service part of the normal multi-user boot.
<i>Alias=dcgm.service</i>	Creates <i>dcgm.service</i> as an alternate name when the unit is enabled.

The process identity and child-process identity are intentionally separate. *User=root* determines the identity of *nv-hostengine* itself. *--service-account nvidia-dcgm* asks the host engine to use the unprivileged *nvidia-dcgm* account for supported child processes. Removing one setting does not replace the other.

Note

User= selects the credentials of the service process; DCGM's *--service-account* separately selects credentials for supported children, which must be able to traverse directories and read or execute every required file. See systemd's [User= documentation](#) and [the Linux process-credentials note](#).

The *-n* option is equally important. It keeps *nv-hostengine* in the foreground. Systemd can then monitor the real server process. If *-n* is removed, the initial process can daemonize and exit while its descendant continues outside the lifecycle that this unit was designed to supervise.

Inspect before editing

Start by asking systemd to display every unit fragment it knows about:

```
$ systemctl cat nvidia-dcgm.service
```

The output labels the source path before each main unit or drop-in. On a system with multi-node diagnostics installed, it can show both the vendor unit and */run/systemd/system/nvidia-dcgm.service.d/50-mndiag.conf*.

Then inspect selected effective properties:

```
$ systemctl show nvidia-dcgm.service \
--property=FragmentPath \
--property=DropInPaths \
--property=ExecStart \
--property=Environment
```

FragmentPath identifies the selected main unit. If it points into */etc*, a full local replacement is already active. *DropInPaths* identifies every drop-in systemd merged. *ExecStart* and *Environment* show the resulting launch command and environment settings rather than just one input file.

Keep a copy of this output when troubleshooting or before changing the unit. It makes it much easier to distinguish a packaged default, a generated value, and a site policy.

Choose a Customization Method

Use the least invasive layer that expresses the intended policy:

Choosing a systemd customization

Method	Appropriate when	Tradeoff
Administrator drop-in	Changing a few directives, adding environment values, or replacing the host-engine command.	Inherits all unrelated vendor settings and future vendor updates. Replacing <i>ExecStart</i> still duplicates that particular vendor command.
Full replacement unit	The site intentionally owns the complete dependency, identity, lifecycle, and installation definition.	Vendor changes are not merged into the local main unit and must be reviewed manually after upgrades.
Edit the vendor unit	Never the appropriate persistent customization method.	A package upgrade can overwrite the change, and the local policy is difficult to distinguish from packaged content.
Edit a file under <i>/run</i>	Never appropriate for persistent local policy.	The file is transient and can be regenerated during boot or a daemon reload.

For most DCGM installations, an administrator drop-in is the right choice.

Customizing with a Drop-In

A drop-in contains only the sections and directives that differ from the selected main unit. The rest of the packaged service remains in effect.

Example: add an environment variable

Suppose an administrator temporarily needs informational host-engine logging. The command-line reference documents `__DCGM_DBG_LVL` as the environment fallback used when `-log-level` is absent.

Open a drop-in:

```
$ sudo systemctl edit nvidia-dcgm.service
```

Enter:

```
[Service]
Environment="__DCGM_DBG_LVL=INFO"
```

When the editor exits successfully, `systemctl edit` saves the file as an administrator drop-in, normally `/etc/systemd/system/nvidia-dcgm.service.d/override.conf`, and reloads the systemd configuration.

At this point the *unit model* has changed, but the running host engine still has its old environment. Inspect and apply the change:

```
$ systemctl cat nvidia-dcgm.service
$ systemctl show nvidia-dcgm.service --property=Environment
$ sudo systemctl restart nvidia-dcgm.service
$ systemctl status nvidia-dcgm.service --no-pager
$ dcgmi discovery --list
```

The final client request matters. A process can be reported as running before the expected listener, libraries, or GPU access are actually usable.

Command-line options take precedence over their environment fallbacks. For example, adding `-log-level ERROR` to `ExecStart` would override `__DCGM_DBG_LVL=INFO`. Use [nv-hostengine](#) to check option and environment precedence before combining them.

Example: replace the host-engine command

A drop-in cannot append words to an existing `ExecStart` command. It must replace the command. The following example preserves the packaged behavior and prevents the Introspection and Profiling modules from loading at startup:

```
[Service]
ExecStart=
ExecStart=/usr/bin/nv-hostengine -n --service-account nvidia-dcgm --denylist-modules 3,8
```

The first, empty *ExecStart=* assignment clears the command inherited from the main unit. Without it, systemd sees more than one start command for a service type that permits only one and rejects the unit. The second assignment defines the complete replacement command.

A replacement must deliberately preserve every packaged option that is still required:

- *-n* or *-no-daemon* keeps the process under systemd supervision.
- *--service-account nvidia-dcgm* preserves the packaged child-process identity.
- The absolute executable path avoids dependence on the service's search path.
- Any locally required listener, logging, home-directory, or module options must appear in the replacement command or environment.

Because this drop-in repeats the vendor *ExecStart*, compare the two after DCGM upgrades. A future package can change the vendor command without changing the local replacement stored under */etc*.

Creating a named drop-in directly

systemctl edit is convenient when one general *override.conf* is sufficient. Sites sometimes use descriptive files so ownership and ordering are obvious, for example:

```
/etc/systemd/system/nvidia-dcgm.service.d/70-site-policy.conf
```

Create or edit the file with the site's normal configuration-management workflow. A direct file edit does not notify systemd, so reload and restart explicitly:

```
$ sudo systemctl daemon-reload
$ systemctl cat nvidia-dcgm.service
$ sudo systemctl restart nvidia-dcgm.service
```

Splitting unrelated concerns into separate files can make later maintenance easier. For example, a site can keep listener policy in *70-listener.conf* and a diagnostic environment in *90-diagnostics.conf*.

Using a Replacement Unit

A full replacement is a local main unit at:

```
/etc/systemd/system/nvidia-dcgm.service
```

Because */etc/systemd/system* has higher priority than */usr/lib/systemd/system*, systemd selects this file instead of the packaged main unit. Use this approach only when the site intends to own the whole unit, not merely to add one host-engine option.

Create a replacement from the current main unit with:

```
$ sudo systemctl edit --full nvidia-dcgm.service
```

The editor opens a complete unit and saves it under */etc*. A replacement could look like this:

```
[Unit]
Description=NVIDIA DCGM service
After=nvidia-gpu-reset.target
Wants=nvidia-gpu-reset.target

[Service]
User=root
PrivateTmp=false
Environment="DCGM_HOME_DIR=/var/log/nvidia-dcgm"
ExecStart=/usr/bin/nv-hostengine -n --service-account nvidia-dcgm
Restart=on-abort

[Install]
Alias=dcgm.service
WantedBy=multi-user.target
```

This example reproduces the packaged unit to illustrate the three sections; it does not require a replacement in practice. A site-owned version might add dependencies, credentials, resource limits, or security controls that are awkward to express as a small drop-in.

A replacement has two maintenance consequences:

1. Package upgrades can update */usr/lib/systemd/system/nvidia-dcgm.service*, but those changes do not modify the main unit under */etc*.
2. Drop-ins still apply. In particular, a generated multi-node diagnostics drop-in under */run* is merged with the replacement unless a higher-priority local drop-in overrides it.

After an upgrade, compare the site unit with the new vendor unit and decide which changes should be incorporated. *systemctl cat* shows which main fragment systemd selected; do not assume the packaged file is active merely because it exists.

Multi-Node Diagnostics Generator

Multi-node diagnostics add one more configuration layer because they launch OpenMPI from the long-running host-engine process. The location of *mpirun* and its libraries varies by Linux distribution. The shell used to invoke *dcgmi mndiag* is not the parent of the diagnostic process, so exports in that shell are not sufficient.

For example, this changes only the client’s environment:

```
$ export DCGM_MNDIAG_MPIRUN_PATH=/opt/openmpi/bin/mpirun
$ dcgmi mndiag ...
```

The *dcgmi* client sends a request to the host engine. The host engine then launches the multi-node work from the environment it received when systemd started it. Unless the variable was also placed in the service configuration and the service was restarted, the export above does not configure that launch.

What the generator does

The multi-node diagnostics package installs:

```
/usr/lib/systemd/system-generators/nvidia-dcgm-multinode-diagnostics-generator.sh
```

Systemd executes system generators early during boot and again during *systemctl daemon-reload*. The DCGM generator performs this sequence:

1. Read *ID* from */etc/os-release*.
2. Select conventional OpenMPI executable and library paths for that distribution.
3. Create */run/systemd/system/nvidia-dcgm.service.d/*.
4. Write *50-mndiag.conf* with the selected environment values.
5. Let systemd merge that drop-in into the effective DCGM service.

The generated file is:

```
/run/systemd/system/nvidia-dcgm.service.d/50-mndiag.conf
```

and has this form:

```
[Service]
Environment="DCGM_MNDIAG_MPIRUN_PATH=<path-to-mpirun>"
Environment="LD_LIBRARY_PATH=<path-to-openmpi-libraries>"
```

DCGM_MNDIAG_MPIRUN_PATH identifies the executable the host engine should launch. *LD_LIBRARY_PATH* is process-wide: the host engine and all of its child processes inherit it. The generator assigns the whole value rather than appending a directory to an existing value.

Paths selected by the generator

Distribution ID	DCGM_MNDIAG_MPIRUN_PATH	LD_LIBRARY_PATH
azurelinux	/usr/lib/openmpi/bin/mpirun	/usr/lib/openmpi/lib

Distribution ID	DCGM_MNDIAG_MPIRUN_PATH	LD_LIBRARY_PATH
<i>debian</i>	<i>/usr/bin/mpirun</i>	Empty
<i>amzn, fedora, rhel, rocky</i>	<i>/usr/lib64/openmpi/bin/mpirun</i>	<i>/usr/lib64/openmpi/lib</i>
<i>sles, suse</i>	<i>/usr/lib64/mpi/gcc/openmpi4/bin/mpirun</i>	<i>/usr/lib64/mpi/gcc/openmpi4/lib64</i>
<i>ubuntu</i>	<i>/usr/mpi/gcc/openmpi-4.1.7rc1/bin/mpirun</i> when that installation is present; otherwise <i>/usr/bin/mpirun</i>	The matching <i>/usr/mpi/gcc/openmpi-4.1.7rc1/lib</i> directory when that installation is present; otherwise empty

The generator selects known paths; it does not search arbitrary OpenMPI installations and does not verify that the chosen executable exists. An unrecognized distribution ID causes it to exit without creating the drop-in. The remaining multi-node requirements—SSH, account access, matching software, network reachability, and node configuration—are outside the generator’s scope.

Why the generated file must not be edited

/run contains runtime state. Its contents do not constitute persistent site configuration, and the generator can rewrite *50-mndiag.conf* during boot or a daemon reload. Editing that file can appear to work until the next regeneration, which makes the failure especially difficult to diagnose.

Place site policy under */etc* instead. The separation is intentional:

- the generator owns distribution-derived defaults in */run*;
- the administrator owns persistent local policy in */etc*; and
- systemd combines them predictably.

Override the generated OpenMPI paths

If OpenMPI is installed under */opt/openmpi*, create:

/etc/systemd/system/nvidia-dcgm.service.d/90-mndiag-local.conf

with:

```
[Service]
Environment="DCGM_MNDIAG_MPIRUN_PATH=/opt/openmpi/bin/mpirun"
Environment="LD_LIBRARY_PATH=/opt/openmpi/lib"
```

The filename sorts after *50-mndiag.conf*, so the local assignments for those variables become the effective values. Then apply and verify them:

```
$ sudo systemctl daemon-reload
$ systemctl cat nvidia-dcgm.service
$ systemctl show nvidia-dcgm.service --property=Environment
$ sudo systemctl restart nvidia-dcgm.service
$ dcgmi mndiag --hostList 'node1;node2' --run mnubergemm
```

Use the host-list form appropriate for the deployment; the exhaustive grammar is in [dcgmi mndiag](#).

Be careful when replacing *LD_LIBRARY_PATH*. It affects more than *mpirun*. Include every library directory required by the host engine and diagnostic children, and validate ordinary DCGM operations after the restart as well as the multi-node diagnostic.

Validate and Troubleshoot

A successful editor exit or *daemon-reload* proves only that systemd accepted a configuration update. A useful validation checks the assembled unit, the new process, and a real DCGM request.

Use this sequence after a change:

```
$ systemctl cat nvidia-dcgm.service
$ systemctl show nvidia-dcgm.service \
  --property=FragmentPath \
  --property=DropInPaths \
  --property=ExecStart \
  --property=Environment
$ sudo systemctl restart nvidia-dcgm.service
$ systemctl status nvidia-dcgm.service --no-pager
$ journalctl --unit=nvidia-dcgm.service --boot --no-pager
$ dcgmi discovery --list
```

Note

journalctl -unit filters records for the service and *-boot* limits the result to the current boot, which helps separate a failed restart from older messages. Access to the system journal can depend on the invoking user's privileges; see the [journalctl manual](#).

The following symptoms usually point to a specific layer:

Common customization problems

Symptom	What to check
---------	---------------

Symptom	What to check
A directly edited drop-in does not appear in <i>systemctl cat</i> .	Confirm the canonical unit name and path, the <i>.conf</i> suffix, and that <i>systemctl daemon-reload</i> was run.
The effective unit shows the change, but the host-engine process does not use it.	Restart the service. A daemon reload does not modify a running process.
Systemd reports more than one <i>ExecStart</i> command.	Add an empty <i>ExecStart=</i> before the replacement command.
The service starts and immediately becomes inactive, or systemd does not supervise the server that remains.	Confirm that the replacement command retained <i>-n</i> or <i>-no-daemon</i> .
A package upgrade changed the vendor unit but service behavior did not change.	Check <i>FragmentPath</i> for a full unit under <i>/etc</i> and inspect drop-ins that replace <i>ExecStart</i> .
Multi-node diagnostics use the wrong <i>mpirun</i> .	Inspect <i>50-mndiag.conf</i> , later local drop-ins, and the effective <i>Environment</i> . Confirm that the service was restarted.
<i>dcgmi</i> cannot connect after a listener customization.	Confirm the effective <i>ExecStart</i> and use the matching <i>-host</i> address. Check the service journal for bind failures.



[Privacy Policy](#) |
 [Your Privacy Choices](#) |
 [Terms of Service](#) |
 [Accessibility](#) |
 [Corporate Policies](#) |
 [Product Security](#) |
 [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.